
Event Handling

Dr. Harish D. Gadade (डॉ. हरिष डी. गडदे)
Dept. of Computer Science and Engg.
COEP Technological University, Pune

Event Handling

- Basics of Event Handling
- Event Handling with Programming Languages
- Comparison of Event Handling and Exception Handling
- Case Study:GUI Development : **Home Assignment**

Basics of Event Handling

◆ What is an Event?

An **event** is an action or occurrence recognized by software.

Examples:

- User clicks a button
- Mouse hovers over an element
- Key is pressed
- Timer expires
- Message or signal is received

◆ Event Handling

Event handling is the process of **detecting events** and **executing corresponding code** (event handler).

An **event handler** (or listener) is a function/method that gets executed when a specific event occurs.

Basics of Event Handling

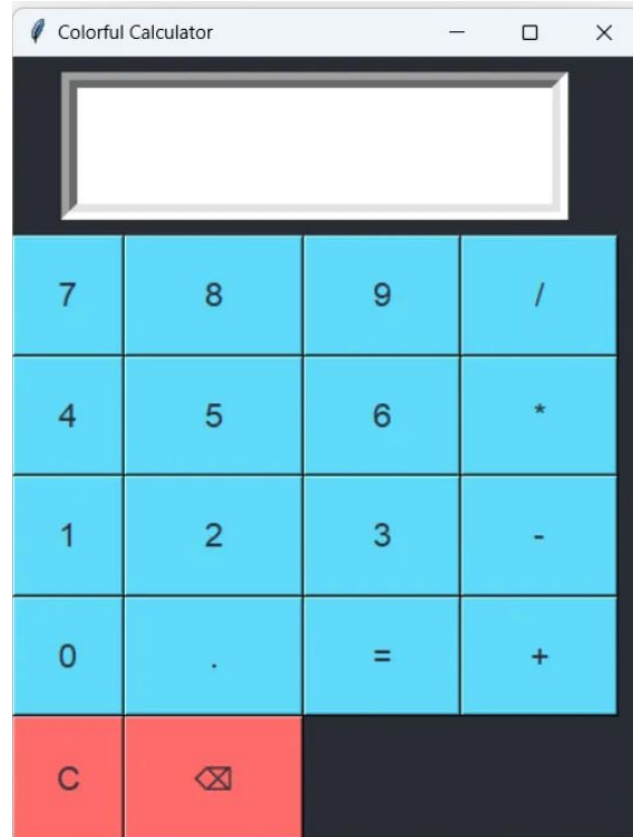
◆ Event Handling Flow

1. **Event Source** – the object that generates the event
(e.g., a button, text field, mouse, keyboard).
2. **Event Object** – carries details about the event
(e.g., which key was pressed, mouse coordinates).
3. **Event Listener (Handler)** – function that executes when the event occurs.

◆ Types of Events

1. **User Events** – mouse click, key press, drag-drop
2. **Window Events** – open, close, resize, minimize
3. **Timer/Clock Events** – scheduled tasks
4. **System Events** – interrupts, signals, network messages

Simple Example



GUI Using Tkinter

- Tkinter is Python's built-in library for creating graphical user interfaces (GUIs).
- Provides a built-in and easy-to-use way to create GUI (Graphical User Interface) applications in Python.
- Offers various widgets like buttons, labels, text boxes and menus to build interactive apps.
- Supports event-driven programming, making it suitable for responsive user interfaces.

Steps to Use Tkinter

To create a Tkinter Python app, follow these basic steps:

1. **Import the tkinter module:** Import the tkinter module, which is necessary for creating the GUI components.
2. **Create the main window (container):** Initialize the main application window using the Tk() class.
3. **Set Window Properties:** We can set properties like the title and size of the window.
4. **Add widgets to the main window:** We can add any number of widgets like buttons, labels, entry fields, etc., to the main window to design the interface.
5. **Pack Widgets:** Use geometry managers like pack(), grid() or place() to arrange the widgets within the window.
6. **Apply event triggers to the widgets:** We can attach event triggers to the widgets to define how they respond to user interactions.

Methods of Tkinter

There are two methods can be used while developing an Application

1. Tk()

To create a main window in Tkinter, we use the Tk() class.

Syntax:

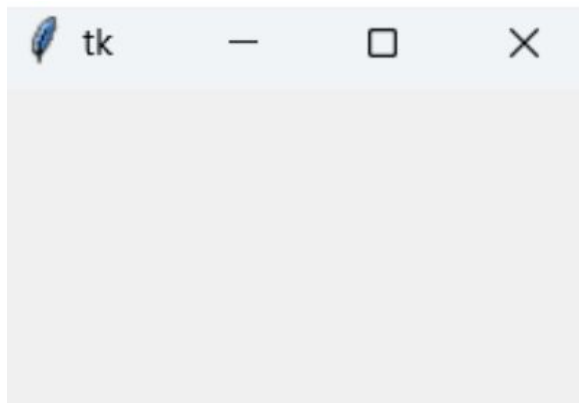
```
root = Tk(screenName=None, baseName=None, className='Tk',  
useTk=1)
```

Methods of Tkinter

2. mainloop()

The `mainloop()` method is used to run application once it is ready. It is an infinite loop that keeps application running, waits for events to occur (such as button clicks) and processes these events as long as window is not closed.

```
import tkinter
m = tkinter.Tk()
'''
widgets are added here
'''
m.mainloop()
```



Tkinter Widgets

1. Label

Syntax

```
w=Label(master, option=value)
```

```
from tkinter import *  
root = Tk()  
w = Label(root, text='COEP Technological University, Pune')  
w.pack()  
root.mainloop()
```

Output

```
COEP Technological University, Pune
```

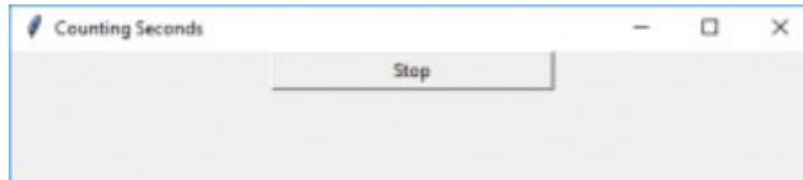
Tkinter Widgets

2. Button

Syntax

w=Button(master, option=value)

```
import tkinter as tk
r = tk.Tk()
r.title('Counting Seconds')
button = tk.Button(r, text='Stop', width=10, command=r.destroy)
button.pack()
r.mainloop()
```

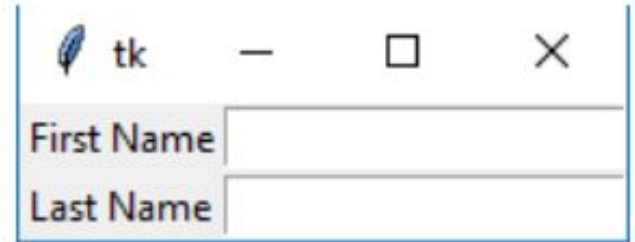


Tkinter Widgets

3. Entry

It is used to input the single line text entry from the user. For multi-line text input, Text widget is used.

```
from tkinter import *
master = Tk()
Label(master, text='First Name').grid(row=0)
Label(master, text='Last Name').grid(row=1)
e1 = Entry(master)
e2 = Entry(master)
e1.grid(row=0, column=1)
e2.grid(row=1, column=1)
mainloop()
```

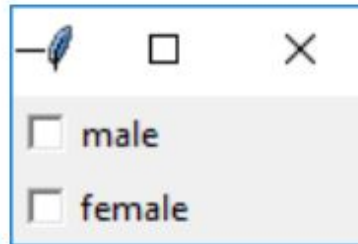


Tkinter Widgets

4. Check Button

A checkbox can be toggled on or off. It can be linked to a variable to store its state

```
from tkinter import *  
master = Tk()  
var1 = IntVar()  
Checkbutton(master, text='male', variable=var1).grid(row=0, sticky=W)  
var2 = IntVar()  
Checkbutton(master, text='female', variable=var2).grid(row=1, sticky=W)  
mainloop()
```

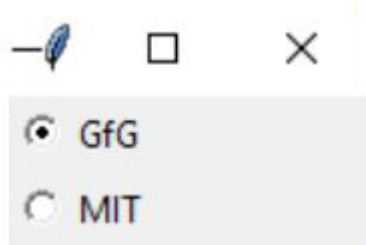


Tkinter Widgets

5. Radio Button

It allows user to select one option from a set of choices.

```
from tkinter import *  
root = Tk()  
v = IntVar()  
Radiobutton(root, text='GfG', variable=v, value=1).pack(anchor=W)  
Radiobutton(root, text='MIT', variable=v, value=2).pack(anchor=W)  
mainloop()
```

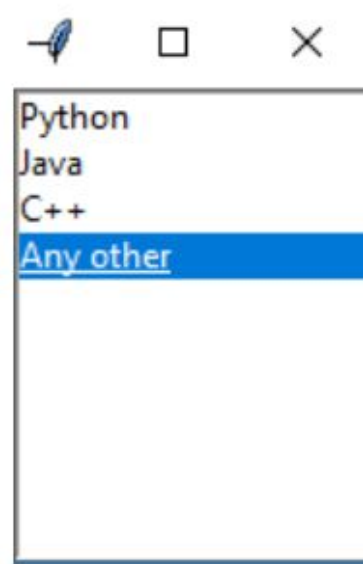


Tkinter Widgets

6. Listbox

It displays a list of items from which a user can select one or more.

```
from tkinter import *
top = Tk()
Lb = Listbox(top)
Lb.insert(1, 'Python')
Lb.insert(2, 'Java')
Lb.insert(3, 'C++')
Lb.insert(4, 'Any other')
Lb.pack()
top.mainloop()
```



Tkinter Widgets

7. Message

It is a widget to display text messages with word wrapping.

Syntax:

```
w = Message(master, option=value)
```

```
from tkinter import *  
main = Tk()  
ourMessage = 'This is our Message'  
messageVar = Message(main, text=ourMessage)  
messageVar.config(bg='lightgreen')  
messageVar.pack()  
main.mainloop()
```



Tkinter Tkinter Geometry Managers

1. Pack() Method

It is a widget to display text messages with word wrapping.

```
import tkinter as tk
```

```
root = tk.Tk()  
root.title("Pack Example")
```

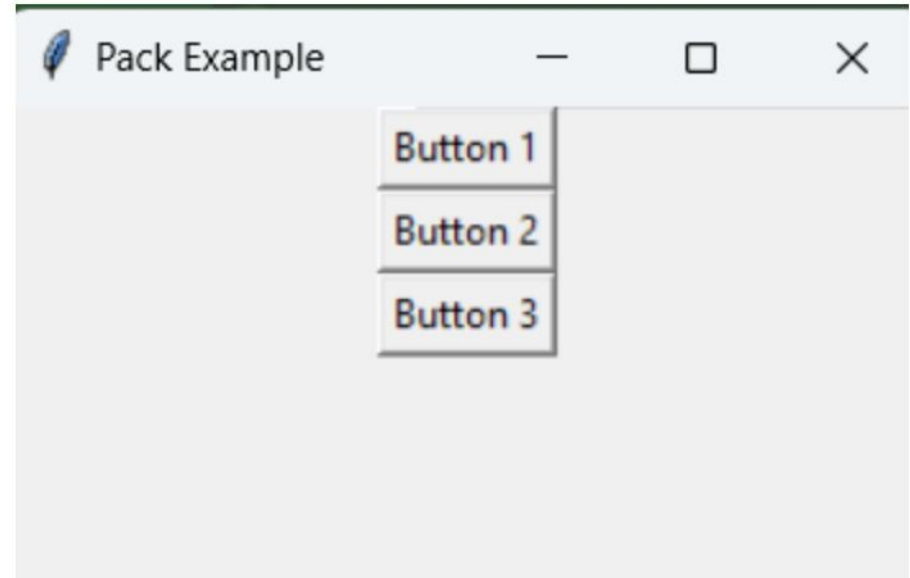
```
# Create three buttons
```

```
button1 = tk.Button(root, text="Button 1")  
button2 = tk.Button(root, text="Button 2")  
button3 = tk.Button(root, text="Button 3")
```

```
# Pack the buttons vertically
```

```
button1.pack()  
button2.pack()  
button3.pack()
```

```
root.mainloop()
```



Tkinter Tkinter Geometry Managers

2. grid() Method

It organizes the widgets in grid (table-like structure) before placing in the parent widget. Each widget is assigned a row and column. Widgets can span multiple rows or columns using rowspan and colspan.

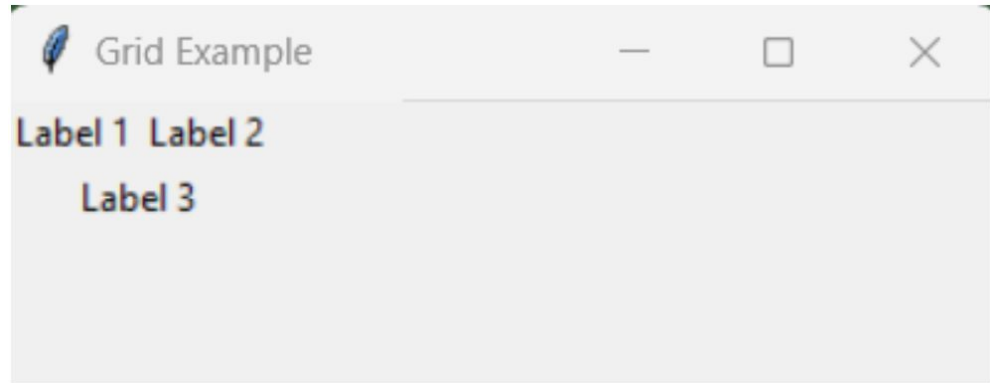
```
import tkinter as tk

root = tk.Tk()
root.title("Grid Example")

# Create three Labels
label1 = tk.Label(root, text="Label 1")
label2 = tk.Label(root, text="Label 2")
label3 = tk.Label(root, text="Label 3")

# Grid the labels in a 2x2 grid
label1.grid(row=0, column=0)
label2.grid(row=0, column=1)
label3.grid(row=1, column=0, colspan=2)

root.mainloop()
```



Tkinter Tkinter Geometry Managers

3. Place() Method

It organizes the widgets by placing them on specific positions directed by the programmer. Widgets are placed at specific x and y coordinates. Sizes and positions can be specified in absolute or relative terms.

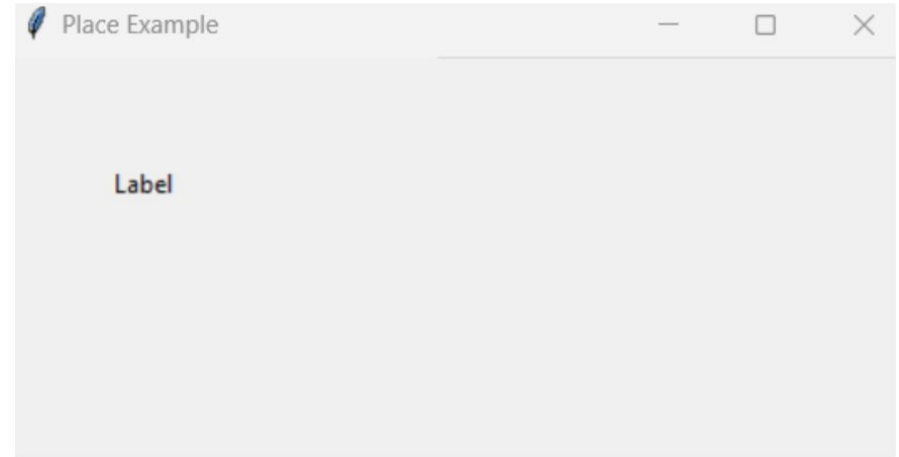
```
import tkinter as tk

root = tk.Tk()
root.title("Place Example")

# Create a label
label = tk.Label(root, text="Label")

# Place the label at specific coordinates
label.place(x=50, y=50)

root.mainloop()
```



Example

```
import tkinter as tk

def on_click():
    print("Button clicked!")

root = tk.Tk()
button = tk.Button(root, text="Click Me", command=on_click)
button.pack()
root.mainloop()
```

Comparison

Aspect	Exception Handling	Event Handling
Trigger	Error/abnormal condition	User/system-generated event
Nature	Reactive (unexpected errors)	Interactive (expected actions)
Control Flow	Alters program flow temporarily	Works alongside main loop
Usage	Reliability and robustness in programs	Interactivity and responsiveness
Example	try-except block	onClick, onKeyPress handlers

Comparison

```
[ Start Program ]
|
v
[ Execute Normal Code ]
|
v
[ Exception Occurs? ]----No----> Continue Program
|
Yes
v
[ Execute Exception Handler ]
|
v
[ Continue Program ]
```

```
[ Start Program ]
|
v
[ Event Loop Running ]
|
v
[ Event Occurs? ]----No----> Wait for Next Event ---> (back to Event Loop)
|
Yes
v
[ Execute Event Handler ]
|
v
[ Wait for Next Event ] ---> (back to Event Loop)
```